A Certificate-First x86-64 Superoptimizer with a Calibrated Comparison Against GCC

Imtamum Hosen Inan

Abstract: **Superoptimization searches for the shortest instruction sequence that implements a function, and its results are usually reported in prose. This paper documents Isuperoptimizer, a seven-week project that treats reporting as part of the experimental apparatus. The system implements a total-semantics model of a curated x86-64 subset, an SMT equivalence miter built on z3, an exhaustive enumerator that emits optimality certificates, and an MCMC search faithful to STOKE's published hyperparameters. Every result is written to an append-only JSONL artifact of record that independent tools re-verify from the log alone. Measured against real gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0 output over the 25-program Hacker's Delight benchmark provenance of Gulwani et al. at both -O2 and -O3, 3 of 23 surviving functions are beaten with z3-verified, x86-encodable, ABI-clean drop-in replacements, including a 4-instruction sign function and an 8-instruction field-exchange. A further 13 functions are matched with exhaustive in-universe optimality proofs totaling 14 216 086 rejected candidates, and 7 survive honestly, three of them after a seeded-synthesis second attempt at five times the primary per-chain synthesis budget. The verification pyramid behind these numbers runs from 66 unit tests through 2 000 000-program differentials and 64 000-case silicon oracles to 114 295 artifact events re-verified with zero failures, and it caught 34 defects, five of them latent model bugs that flawless testing had hidden. The methodological result is that when every claim must survive artifact inspection, the artifacts become trustworthy by construction.**

Index Terms: **superoptimization, program verification, SMT solving, x86-64, compiler optimization, reproducible research**

3/23
functions beaten
with verified drop-ins

13/23
matched with
optimality proofs

114 295
events re-verified
zero failures

34
defects caught by
the pyramid

I Introduction

SUPEROPTIMIZATION is the search for the provably shortest instruction sequence implementing a function. Since Massalin's superoptimizer [1], the field has produced instruction sequences that neither compilers nor textbook authors had found. Such results are usually reported in prose: a paper asserts that a sequence is shorter, equivalent, and optimal, and the reader trusts the authors, the test suite, and the solver.

This project treats that trust as part of the measurement problem. Weeks of implementation produce search results, but they also produce transcription errors, verifier-adjacent bugs, self-neutralizing tests, and model abstractions that license behavior the hardware does not have. Isuperoptimizer was built around one rule: no claim is ever retyped from memory. Every headline number is a machine-extracted property of an append-only JSONL artifact, independent binaries re-derive every concretely checkable claim from the log alone, and an external claims audit recomputes the paper's numbers from the committed files. The build that produced this PDF first re-verified 114 295 artifact events with zero failures and re-ran the 485-check audit.

I-A Contributions

- **A certificate-first superoptimizer** for a curated x86-64 subset: total semantics with per-flag definedness, a z3-backed equivalence miter, an exhaustive backward-liveness enumerator that rejects every shorter candidate with a logged witness, and an MCMC search faithful to STOKE's published move set and hyperparameters [4] (Section III).
- **A seven-layer verification pyramid** in which no layer trusts another: unit tests, millions-scale concrete-vs-symbolic differentials, solver-sampled round-trips, a self-verification grid, hand-picked equivalence and broken pairs, a silicon hardware oracle, and artifact re-derivation (Section IV).
- **A calibrated compiler comparison at benchmark scale**: 3 beaten, 13 matched with proof, 7 honestly exhausted, over 23 real compiler-emitted functions from the Gulwani/STOKE provenance [3], [4] at both optimization levels (Section V).
- **A 34-row defect ledger** with a failure-shape taxonomy, every row recording what caught the defect and why the other layers could not (Section VI). Five defects were found by reading our own claimed winners against the ISA and the ABI; the ledger records that the wins reported here survived three adjudication rounds in which two earlier headline results were retracted for licensing non-encodable instructions and callee-saved register writes.

II Background and Related Work

Manuscript prepared November 2026. Copyright © 2026 Imtamum Hosen Inan. The complete project record (11 architecture decision records, a 34-row verification ledger, and nine machine-checkable JSONL artifacts of record) is committed alongside the source in the Isuperoptimizer repository. Every number printed in this paper was machine-extracted from those artifacts by `extract_data.py`; the build refuses to complete if any claimed value cannot be traced.

Independent research project.

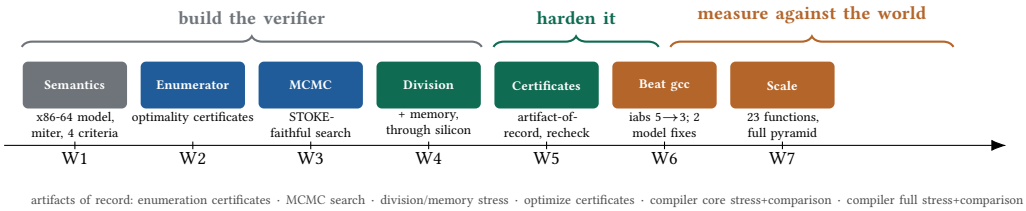


Fig. 1. The seven-week arc. Weeks 1 to 4 built the semantics layer, miter, enumerator, stochastic search, and the division/memory extensions; Week 5 turned reporting itself into a verified pipeline after two incidents (Section VI); Weeks 6 and 7 pointed the tool at real compiler output and scaled the comparison to the full benchmark provenance. One artifact of record was committed in each week that produced results, and all nine remain re-checkable today. Theme labels follow the part structure of the project's research log (Parts I to VII).

II-A Superoptimization and program synthesis

Massalin's superoptimizer [1] exhaustively enumerated instruction sequences over a toy RISC ISA and found provably minimal programs. Denali [2] scaled the approach with goal-directed enumeration. Peephole superoptimizers [9] enumerate short instruction sequences to replace compiler-generated patterns, the same machinery carried to binary translation in [10], and Souper [11] applies solver-based synthesis to compiler IR, where the rewrite search runs over SSA rather than machine code. The synthesis literature supplies the broader framing: sketch-based combinatorial synthesis [19], syntax-guided synthesis [18], solver-aided host languages [20], inductive synthesis frameworks [21], and general surveys [22]. Chlorophyll [17] uses synthesis to compile for low-power spatial architectures.

Gulwani et al. [3] formulated loop-free synthesis as an SMT constraint problem and introduced, in its evaluation, a 25-program benchmark suite drawn from Warren's Hacker's Delight [5]. The programs run from P1 (turn off the rightmost 1-bit) to P25 (compute the high half of a product). STOKE [4] adopted the same 25 programs; its Figure 11 runs them as p01 to p25 and reports synthesis timeouts and algorithmically distinct rewrites. This paper scales the compiler comparison to that full set.

II-B Stochastic superoptimization

STOKE [4] reformulated superoptimization as stochastic search over a fixed-length program space: MCMC with four proposal moves (opcode, operand, swap, instruction), an "improved equality" metric eq' that scores register-file Hamming distance against a testcase set, and a correctness check deferred to an SMT solver. Its published hyperparameters ($p_{\text{opcode}} = 0.16$, $p_{\text{operand}} = 0.5$, $p_{\text{swap}} = 0.16$, $p_{\text{instruction}} = 0.16$, $\beta = 0.1$, 32 testcases) are the ones our search implements; every deviation is documented in the project's ADR-007 and logged in each run's artifact. Schkufza et al. extended the approach to floating-point programs with tunable precision [16].

II-C Verification of rewrites

A parallel tradition verifies individual rewrites instead of searching for them. Translation validation [12], [32] checks each compiler output against its input, equivalence-modulo-inputs testing [13] validates compilers by differentially executing mutated program variants, and Alive2 [14] performs bounded translation validation for LLVM IR. Verified compilation [33] and formalized IR semantics [35] pursue the same goal from the other direction. Our miter applies the same contract at instruction granularity, and our claims are certificates over a declared universe that can be re-derived from committed logs.

II-D Differential testing and compiler validation

Differential testing [38] compares independent implementations on shared inputs. Randomized differential testing served as a prelude to formal verification [39], and property-based testing [42] established the generator-and-oracle pattern that our bulk differential implements. Compiler testing at scale produced Csmith [34], test-case reduction [40], and fuzzer scheduling work [41]. These tools test compilers as wholes. Our pyramid applies the same differential idea inside a single function's instruction semantics, at millions of evaluations per run.

II-E Reporting discipline

The project's distinguishing concern, that the report is part of the experimental apparatus, follows two failures we document rather than cite: a mis-transcribed counterexample in a prose report (ledger row 22), and a genuine discovery whose evidence was destroyed by the debugging workflow around it (row 23). Both incidents created the artifact-of-record rule under which this paper is built: claims live in append-only logs, tools re-derive them, and prose cites them.

III System Architecture

The toolchain is five components, each independently tested, connected only by the artifact of record.

Semantics layer. A total-semantics model of a curated 64-bit x86-64 subset: `movq`, `movabsq`, ALU and shift forms, `leaq`, `setcc`/`cmovcc`, `negq`/`notq`, two-operand `imulq`, one-operand `divq`/`idivq`, and load/store over a 256-byte sandbox. Instruction semantics follow the Intel SDM [8] and the AMD64 programmer's manual [44]; the SysV ABI [7] fixes the calling convention, and the cost model's latency estimates follow Agner Fog's tables [45]. The v0 partial-register ban (ADR-001) rejects every non-64-bit register operand as a hard parse error, never a silent truncation. Every instruction carries both concrete and symbolic semantics, including per-flag definedness: the four RFLAGS bits are modeled as values with

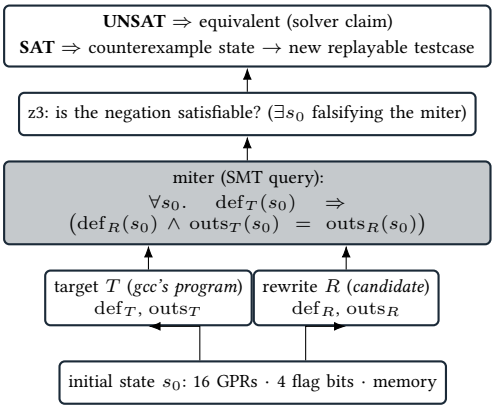


Fig. 2. The equivalence miter as a block diagram. The rewrite must be defined wherever the target is defined and must match every live-out register; the solver's counterexample models become replayable test cases logged in the artifact. Structure per the project's ADR-003 (miter contract).

definedness predicates, so reading a flag nobody wrote is a definedness violation rather than a silent value (Fig. 2). Full-ISA formal semantics exist [15], [36], and the x86-TSO model [37] covers the memory-ordering side that our straight-line subset does not model. Our subset is deliberately narrower: total, differentially tested against silicon, and small enough that every instruction's symbolic encoding is audited.

Equivalence verifier. The miter asks z3 [6] whether

$$\forall s_0.\ \mathrm{def}_T(s_0) \Rightarrow \big(\mathrm{def}_R(s_0) \wedge \mathrm{outs}_T(s_0) = \mathrm{outs}_R(s_0)\big) \quad (1)$$

for target T and rewrite R: the rewrite must be defined wherever the target is, and the live-out registers must agree. UNSAT means equivalence; a satisfiable model is a concrete counterexample state that becomes a replayable test case. A definedness hole in the rewrite fails verification, because $\mathrm{def}_T \Rightarrow \mathrm{def}_R$ is part of the formula rather than an afterthought. Queries are QF_AUFBV and travel over the SMT-LIB 2.0 interface [43]. The bit-vector [25] and array [26] theories involved are standard, the decision-procedure background is covered by [30], [31], and the DPLL(T) architecture [27], [28] underlies the solver line [23], [24], [29] in which such queries are decided.

Enumerator. Exhaustive search over a declared universe (register sets, immediate pools, shift counts, addressing modes, all logged in an `enumeration_universe` event) with backward liveness pruning. An optimality certificate is the residue of the sweep: every candidate shorter than the optimum is rejected with a concrete divergence witness, an input state on which candidate and spec disagree, logged one by one (with a count-summarized threshold for large classes).

MCMC search. Chains over the fixed-length program space with STOKE's four moves and a static per-opcode latency table. Equivalence is consulted only for zero-eq′ candidates (the "eq* discipline"), with short per-query timeouts inside chains and a long-timeout re-verification before any claim is logged.

Artifact pipeline. Every run writes an append-only JSONL log. `recheck` re-derives every concretely checkable claim from the log alone (testcases, rejection witnesses, optima, counterexample pairs) and prints a per-kind tally so its totals decompose by construction. `audit_claims.py` recomputes the headline numbers and enforces the two calibration contracts of Section V-D on every claimed program.

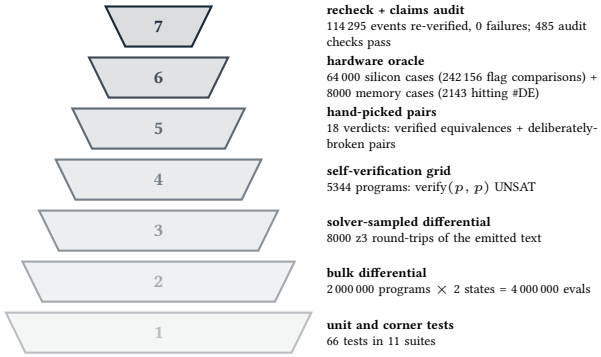


Fig. 3. The verification pyramid, with current volumes from `stress-compiler-full.jsonl` mode summaries, the live `recheck` sweep, and the source tree's test count. No layer trusts another: the concrete evaluator is checked against silicon (6), the symbolic evaluator against the concrete (2, 3), the miter against itself (4), hand-verified mathematics against all of the above (5), and the reporting against the artifacts (7), with 114 295 events re-verified and 0 failures at the start of this document's build. Layer definitions follow the project's verification document.

IV The Verification Pyramid

Figure 3 shows the seven layers and the volume at which each currently runs. Each layer has a characteristic blind spot that the ledger makes concrete, which is why none of them is trusted alone.

Concrete-vs-symbolic differentials (layers 2 and 3) catch semantic disagreement. Failures of the form "the fold is right, the printer is wrong" (rows 3, 13, and 14 of the ledger) stay invisible to them, because the constant-folded comparison never round-trips through the emitted solver text. Layer 3 closes that gap by having z3 evaluate the printed SMT-LIB2.

Random sampling catches large formula errors (row 11 recorded 1,733 discrepancies in one run) but has essentially zero probability at exact boundaries. Row 15 required one hit in 8000 samples, and row 17 was caught by a hand-picked pair on its first run.

The hardware oracle checks the model against silicon, including a #DE pre-check protocol for division and a flag-preset protocol for instructions that leave flags untouched. That protocol checks RFLAGS preservation on real silicon as well as in the model.

Artifact re-derivation (layer 7) catches failures that no program test can express. Its catches include a verifier that counted events its own documentation described as echoed, and an audit script that printed its verdict before checking the sections that mattered (rows 28 and 34).

V The Compiler Benchmark

V-A Protocol

The week-7 comparison targets the full provenance of Figure 4 of [3]: all 25 P-programs, ported to `uint64_t`/`int64_t`

C. Word-size scaling decisions and two corrections of the paper's own transcription slips, each verified against brute force, are documented in the committed sources. Six further functions carried over from the project's sixth week. All 31 functions were compiled by gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0 at -O2 and -O3 with `-m64 -fno-tree-vectorize -fno-tree-slp-vectorize`; the extractor for this paper asserts that the two levels emit byte-identical assembly for every function before the document builds. Both levels are nevertheless logged and challenged separately: 46 `compiler_baseline` events, each carrying the compiler version, flags, C source, and emitted assembly verbatim.

Reconciliation against the instruction subset left 23 survivors. The eight exclusions (boolean tests compiled to `setcc-on-%al` plus `movzbl`, a `testq`-based power-of-two test, the `xorl` zeroing idiom inside an otherwise all-`cmov` three-way cycle, and 32-bit narrowing in the multiply-high decomposition) are each logged with the actual parse error their assembly produces: sixteen `exclusion_confirmed` events, one per function per level. The two opcodes the batch forced into the subset, `notq` and the `salq` spelling of `shlq`, received the full pyramid treatment of Section IV with no exceptions.

V-B Results

Figure 4 shows the headline: of 23 functions attempted end-to-end, gcc is beaten on 3, matched with exhaustive proofs on 13, and survives honestly on 7. Table I carries the per-function detail. Every outcome class is machine-derived: the extraction script reproduces the audit's derivation, and the ratio is a property of the committed artifact.

The three wins. Figures 5 to 7 show the verified drop-in replacements beside gcc's own code, exactly as logged in the artifact. Each carries a final long-timeout z3 equivalence verdict. The iabs rewrite is the one the project reported in its sixth-week comparison; the other two are new at scale.

Optimality certificates at scale. The 13 matched functions are proofs rather than absences of results. The enumerator exhaustively rejected every shorter program in each declared universe: 14 216 086 candidates in total across the 26 certificates of the batch (13 functions at both levels, plus the week-2 suite's own 218 955), each rejection backed by a logged divergence witness. Figure 8 shows the per-function scale of those sweeps, spanning four orders of magnitude: 472 candidates to certify sign_mask minimal, and 5 104 879 for sat_add.

V-C The stragglers' second attempt

Three functions had survived their search budgets in the project's sixth week without improvement. The protocol requires a serious second attempt rather than a silent repeat, so each received seeded synthesis: 8 chains of 300 s per level, against a primary pass whose logged chain wall-clock medians are 120 s (optimization) and 60 s (synthesis). The second attempt is therefore 2.5 times the primary optimization and five times the primary synthesis budget per chain. Chains start from structural variations of the compiler's own form (the identity, single-instruction deletions, and adjacent swaps) rather than from random programs. The rationale and per-chain starts are logged in `straggler_plan` and `seeded_start` events. Table II records the outcome from the logged summaries: no verified improvement from informed starts either. gcc's forms for these three survive the second attempt at both levels, and that is reported as the result.

V-D Calibration: making the comparison mean something

Two of the three wins above were not the first wins found. The difference between them records what the calibration pass changed. The first full-scale run claimed the popcount program with a 14-instruction rewrite, and the second run claimed the sign function with a 4-instruction one. Reading those claimed winners against the real machine, before reporting either, showed that both licensed forms the hardware does not have.

The popcount winner used `andq` with 64-bit immediates in three of its forms. Real ALU instructions take only sign-extended 32-bit immediates, which is why gcc had materialized those constants with `movabsq`. As actual instructions the winner was 17 programs long, worse than gcc. The search had instruction forms that the compiler's output structurally cannot contain, so the comparison was weighted in our favor.

The sign-function winner held its -1 in `%rsp`. In the model, where all sixteen registers are equal and the live-out is RAX, this is equivalent, and the solver proved it. A real function body that sets RSP returns through the wrong address. Equivalent programs and equivalent functions diverge exactly when a candidate writes a callee-saved or stack register.

Both were fixed to make the two notions of equivalence coincide by construction. The searched subset now admits only imm32-encodable ALU immediates (full-width constants must be materialized via `movq`, as gcc does) and only caller-clobbered register destinations. The model itself stays permissive, because committed rejection witnesses from earlier universes replay through it and must keep re-checking. The comparison was then re-adjudicated from scratch. The popcount win vanished (the function is now an honest no-improvement), while the sign-function win was found again, cleanly, as a different 4-instruction construction using a scratch register (Fig. 6). The claims audit enforces both contracts on every claimed program, so the calibration is checked rather than remembered. The week-2 certificate suite was regenerated through each universe change: 278 377 candidates, then 279 088, then 249 223, then 218 955, with byte-identical optima each time.

VI Lessons Learned

The ledger (Table III) records the project's defects and the layer that caught each one. Its rows cluster into failure shapes.

The story is not the artifact. Rows 22 and 23 are the founding incidents: a correct machine result mis-transcribed into a report, and a discovery whose evidence was destroyed by the debugging workflow around it. External verification of claims the pipeline had already blessed found both. They produced

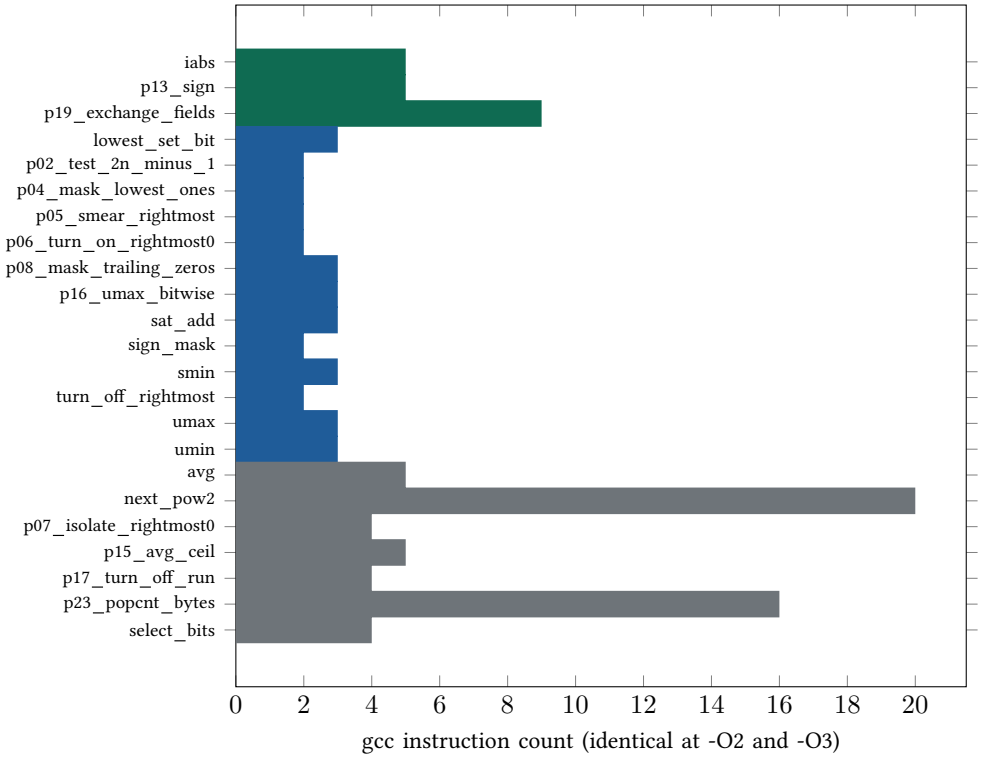


Fig. 4. Outcome of the compiler comparison for all 23 functions, ordered beaten, matched, honest; bar length is gcc's instruction count for that function. Generated from `compiler-comparison-full.jsonl` (compiler_baseline lengths; outcome derivation identical to `audit_claims.py`). All three wins are verified drop-in replacements, x86-encodable and touching only caller-clobbered registers; the seven honest no-improvements survived MCMC budgets, and three of them additionally survived the seeded second attempt of Section V-C.

TABLE I

PER-FUNCTION OUTCOMES OF THE COMPILER COMPARISON. "GCC" IS THE INSTRUCTION COUNT OF THE EMITTED BASELINE (IDENTICAL AT BOTH LEVELS); "RESULT" IS THE VERIFIED SHORTER LENGTH WHERE ONE EXISTS. CERTIFICATE SCALE IS THE EXHAUSTIVE ENUMERATION'S REJECTED-CANDIDATE COUNT AND SOLVER QUERIES (-O2; IDENTICAL AT -O3 WHERE THE UNIVERSE MATCHES), OR THE SEARCH OUTCOME FOR MCMC-SCOPED FUNCTIONS. EVERY ROW DERIVES FROM `compiler-comparison-full.jsonl` EVENTS (`compiler_baseline`, `enumeration_summary`, `optimum`, `search_result`); THE TABLE BODY IS EMITTED BY `extract_data.py`, NOT TYPED.

Function	Ref.	gcc	outcome	result	certificate scale / evidence
iabs	P9	5	beaten	3	perf 5 → 3 via MCMC (z3-equivalent)
p13_sign	P13	5	beaten	4	perf 5 → 4 via MCMC (z3-equivalent)
p19_exchange_fields	P19	9	beaten	8	perf 9 → 8 via MCMC (z3-equivalent)
lowest_set_bit	P3	3	matched	3	871,298 cand., 6 z3
p02_test_2n_minus_1	P2	2	matched	2	1,715 cand., 1 z3
p04_mask_lowest_ones	P4	2	matched	2	2,695 cand., 1 z3
p05_smear_rightmost	P5	2	matched	2	2,205 cand., 1 z3
p06_turn_on_rightmost0	P6	2	matched	2	2,205 cand., 1 z3
p08_mask_trailing_zeros	P8	3	matched	3	555,680 cand., 1 z3
p16_umax_bitwise	P16	3	matched	3	139,768 cand., 1 z3
sat_add	n/a	3	matched	3	5,104,879 cand., 2 z3
sign_mask	n/a	2	matched	2	472 cand., 1 z3
smin	n/a	3	matched	3	146,108 cand., 3 z3
turn_off_rightmost	P1	2	matched	2	1,715 cand., 1 z3
umax	n/a	3	matched	3	139,507 cand., 1 z3
umin	n/a	3	matched	3	139,796 cand., 1 z3
avg	P14	5	no improve	5	MCMC budget, no verified improvement
next_pow2	P24	20	no improve	20	MCMC budget, no verified improvement
p07_isolate_rightmost0	P7	4	no improve	4	MCMC budget, no verified improvement
p15_avg_ceil	P15	5	no improve	5	MCMC budget, no verified improvement
p17_turn_off_run	P17	4	no improve	4	MCMC budget, no verified improvement
p23_popcnt_bytes	P23	16	no improve	16	MCMC budget, no verified improvement
select_bits	n/a	4	no improve	4	MCMC budget, no verified improvement

the artifact-of-record rule that everything else here descends from.

The fold is right, the printer is wrong. Rows 3, 13, and 14 share a shape: the constant-folded semantics agreed with concrete

gcc, 5 instr.

```
movq %rdi, %rax
sarq $63, %rax
xorq %rdi, %rax
shrq $63, %rdi
addq %rdi, %rax
```

verified, 3 instr.

```
andq $0, %rax
subq %rdi, %rax
cmovleq %rdi, %rax
```

Fig. 5. iabs (P9, $|x|$): gcc computes the sign mask twice. The verified rewrite zero-builds the accumulator, subtracts x (setting flags), and selects x when $-x \leq 0$. Flags are read only after `subq` writes them, which is sound under the corrected entry-flags model (ledger row 24). From `compiler-comparison-full.jsonl` (`compiler_baseline`, `search_result`), perf 5 to 3.

gcc, 5 instr.

```
movq %rdi, %rax
sarq $63, %rdi
negq %rax
shrq $63, %rax
orq %rdi, %rax
```

verified, 4 instr.

```
orq $0xffffffffffffffff, %r10
addq %rdi, %rdi
seta %rax
cmovbq %r10, %rax
```

Fig. 6. p13_sign (P13, the HD sign function): doubling x in `addq %rdi,%rdi` makes CF exactly the original sign bit, since doubling overflows if and only if the MSB was set. `seta` yields 1 for strictly positive x (CF$=0$ and ZF$=0$), and `cmovbq` selects -1 when CF is set. At $x = \text{INT64_MIN}$ the double wraps to zero (ZF set) while CF remains set, so the two predicates still partition the sign cases correctly. From `compiler-comparison-full.jsonl` (`search_result`, phase optimization), perf 5 to 4.

gcc, 9 instr.

```
movq %rdx, %rcx
movq %rdi, %r8
shrq %cl, %rdi
xorq %r8, %rdi
andq %rsi, %rdi
movq %rdi, %rax
xorq %rdi, %r8
shlq %cl, %rax
xorq %r8, %rax
```

verified, 8 instr.

```
movq %rdi, %rax
movq %rdx, %rcx
shrq %cl, %rdi
xorq %rax, %rdi
andq %rsi, %rdi
xorq %rdi, %rax
shlq %cl, %rdi
xorq %rdi, %rax
```

Fig. 7. p19_exchange_fields (P19, HD's field swap with runtime distance k): gcc parks x in two places, and the rewrite holds it in `%rax`/`%rdi`, saving the copy. Both forms use the variable `%cl` shifts the subset has carried since its first week. From `compiler-comparison-full.jsonl` (`search_result`), perf 9 to 8.

TABLE II
THE STRAGGLERS' SECOND ATTEMPT (SEEDED SYNTHESIS, 8 CHAINS × 300 S PER LEVEL). FROM `straggler_summary` EVENTS IN `compiler-comparison-full.jsonl`.

benchmark	spec perf	best found	status
avg (-O2)	5	5	no_improvement
avg (-O3)	5	5	no_improvement
next_pow2 (-O2)	20	20	no_improvement
next_pow2 (-O3)	20	20	no_improvement
select_bits (-O2)	4	4	no_improvement
select_bits (-O3)	4	4	no_improvement

evaluation everywhere, while the emitted solver text was wrong. Concrete-vs-symbolic testing cannot see this, because it never round-trips through the printed text. This is why layer 3 of the pyramid exists.

A sound model is not a well-tested model. Rows 24, 32, and 33 are the calibration incidents of Section V-D, together with the entry-flags correction that compiler integration forced in week 6. Each is a model in which every test passes because the tests share the model's blind spot. Each was found only by confronting the model with something outside it: real compiler output, the ISA's encoding rules, or the calling convention.

Tests that cannot fail. Row 29 records a documented opcode acceptance whose pinning test contained a fallback that made it pass exactly when the claim was false. The ADR said it worked, the test said it worked, and the parser rejected every real instance. The un-neutered form of the test found the bug the same day the batch first emitted the opcode.

The verifier needs verifying. Rows 28 and 34 live in the observability layer: an event counter whose totals did not decompose to what it validated, and an audit script that printed its verdict before running the sections that mattered. The ledger's throughline, stated at the project's week-6 close and kept verbatim since: *a sound model and a well-tested model are different things, and "every test passes" is a claim about the model, not the world.*

The distribution of catchers argues for the pyramid's shape. Unit tests caught corner arithmetic that the random layers rarely reach. The bulk differential caught the release-mode panic that weeks of corner tests had missed (row 27). Solver sampling caught printer bugs. A hand-picked pair caught a one-in-2^{64} boundary that 8000 random samples had never touched. Silicon caught no semantics bugs, because the model was right, but its #DE protocol caught oracle bugs instead (row 7). The artifact layer caught the reporting failures that no program-level test can even express.

VII Limitations

The claims of this paper are scoped, and the scopes are load-bearing.

- **Optimality is relative to a declared universe.** The enumerator proves that no shorter equivalent exists among the declared registers, immediates, and forms; the universe is logged with every certificate. No global claims are made, and the 13 matches are matches within those universes.
- **Straight-line code only.** Branches and loops are excluded at the parse layer, where a local label is a hard error. The project's own week-6 hand-written cycle function was excluded for this boundary. No week-7 exclusion was control-flow, but that is fortune rather than reach: gcc happened to compile the entire batch branchlessly, and any function it compiles with a branch falls outside the model.
- **64-bit registers only.** The partial-register ban, the project's oldest rule, excluded seven of the eight week-7 non-survivors (booleans compiled through `setcc` on `%al` and `movzbl`, and the `xorl`/`andl`/`movl` narrowing idioms). The eighth, the power-of-two test, fell to `testq`, a flags-only opcode outside the subset. All eight exclusions are empirically confirmed against the committed assembly.
- **Equivalence is a solver claim.** z3's UNSAT verdicts are labeled as solver claims throughout. `recheck` validates everything around them (testcases, witnesses, cost-zero on committed sets) and never pretends to concretely re-check an equivalence.

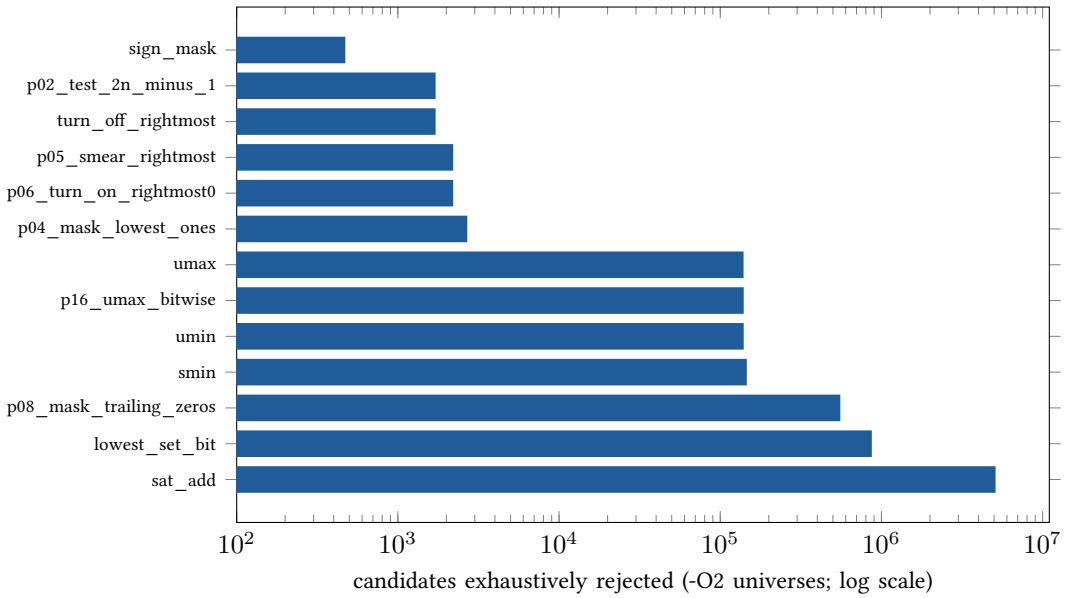


Fig. 8. The scale of the optimality certificates for the 13 matched functions: the number of candidate programs the enumerator evaluated and rejected, each with a logged witness, before concluding that no shorter equivalent exists in the declared universe. Generated from `enumeration_summary.candidates_total` events in `compiler-comparison-full.jsonl` (-O2).

- **Latency is a static table.** The perf metric is a per-opcode latency table rather than measured cycles; the instruction count comparisons shown for the wins are count-level claims.
- **One compiler, one machine.** gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0 is the only compiler compared, and the comparison would need re-running per compiler release. Both optimization levels were compared despite emitting identical code, which is logged rather than assumed.

VIII Future Work

Two exclusions dominate the loss rate at scale, and they are the next frontiers.

Partial-register support. The 32-bit and 8-bit sub-register semantics that the v0 ban rejects are well-defined on real hardware. Modeling them soundly (zero-extension on write, the `%al`-within-`%rax` aliasing, and the x86-64 partial-register stalls) would return eight benchmark functions to the comparison, including the three-value cycle whose emitted code is otherwise entirely `cmov`s. The cost is real: every new opcode form in this project earned a full pyramid treatment before it could appear in any claim, and sub-register semantics multiply the aliasing surface of every existing test.

Control flow. The straight-line model excludes branches by design. The `je`-shaped exclusions, and every real-world function with a loop, need a bounded-control-flow model: basic blocks with structured merge points, or the loop-free-program formulation of [3] itself, before the comparison can leave the Hacker's Delight genre. The certificate infrastructure (witnesses, replay, recheck) transfers unchanged; the miter does not.

Beyond these, three directions follow from the current design. Learned latency from hardware counters would replace the static table, as STOKE's original methodology did [4]. Testcase broadcasting across parallel chains would improve search efficiency. Extending the compiled-binary oracle to the new frontiers would repeat, for control flow and partial registers, what the project's fourth week did for division and memory.

IX Conclusion

This paper reports a superoptimizer that beats a production compiler on 3 of 23 real benchmark functions with verified, encodable, ABI-clean drop-in replacements, proves 13 more optimal within declared universes totaling 14 216 086 rejected candidates, and reports honestly on the rest, three of them after a five-times-budget informed-start second attempt. The center of the work is the discipline that makes those numbers meaningful: a seven-layer pyramid in which every layer assumes the others are wrong, an artifact pipeline that re-derives every claim from committed logs (and re-verified 114 295 events with zero failures as the first step of building this document), and a 34-row ledger that records, in the project's own voice, every way the story diverged from the artifact until the pipeline made the divergence impossible to hide.

The benchmark result and the ledger describe one finding. Search finds what compilers miss, and reporting discipline is what makes the finding real. Inspecting the artifact is what separates a result from a claim.

References

[1] H. Massalin, "Superoptimizer: A look at the smallest program," in *Proc. Second Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, 1987, pp. 122–127.

TABLE III

A selection of the 34-row defect ledger. The thirteen rows shown are selected for instructiveness, and the text is abridged to first sentences; the full ledger is a committed project document. Every row is machine-extracted from that document by `extract_data.py`, so the selection, not the content, is editorial.

#	bug	caught by	why the other layers missed it
1	Concrete IMUL zero-extended operands into i128 ; wrong CF/OF for negative operands	corner unit test	random states hit negative×negative rarely in the flag bits that differ; symbolic construction was *right*, so differential testing would eventually catch it but the corner test did first
3	`msb()/bit()` compared a 1-bit extract against a 64-bit constant ; z3 sort error	miter self-checks (grid)	all *folding* was correct; only the printed SMT was wrong, so layers 2/6 saw nothing
8	Enumerator optimized the program *as written* and exposed a desc/spec mismatch (`shl $63` extracts bit 0, not 63)	the enumerator itself (enumeration layer)	the spec was wrong; no test layer can catch an author's intent error ; the certificate is about the program text, which is why the tool was right to optimize the text
11	Unsigned #DE bound printed as the 128-bit all-ones literal instead of zero-extended u64::MAX ; every quotient $\leq 2^{128}$ counted as defined	z3-sampled differential (1,733 discrepancies)	the *fold* was correct; concrete/fold differential (layer 2) saw perfect agreement
13	Load64 printer byte-rotated the concat chain (s6..s0,s7)	z3-sampled differential (384 cases)	folds were right AND the miter stayed sound (both sides rotate identically) ; only cross-checking z3's evaluated memory against concrete bytes exposed it
15	Signed #DE lower bound used `bvsgt` ; the exact INT64_MIN quotient boundary was misclassified	z3-sampled differential (1 case in 8,000)	a one-in-2^{64}-ish boundary; random sampling hit it once
17	Signed #DE **upper** bound used `bvslt` ; quotient exactly i64::MAX (defined) rejected as undefined; the miter "counterexampled" a pair whose outputs were equal.	**hand-picked broken pair, first run** (division/memory review-gap closure)	survived 8,000 random z3 samples *and* the earlier boundary fix's symmetric twin: random inputs essentially never make the quotient exactly i64::MAX.
22	An early prose report mis-transcribed a counterexample input	**external hand-verification**	the machine outputs were right; the *report* was the bug ; this created the artifact-of-record pipeline (ADR-009)
23	The i64::MAX boundary bug's discovery witness (row 17) was **destroyed**: the failing run's artifact was overwritten by the truncating re-run and then `rm`'d before the commit.	external audit of the *audit machinery*	nothing in the pyramid watches the debugging workflow itself; failure evidence produced outside the artifact pipeline has no protection.
24	Model: entry flags treated as defined-false ; candidates could read RFLAGS at function entry and get fake constants (`sete %rax` at entry → constant 0).	**compiler-output integration** (compiler-baseline phase)	every earlier layer used hand-written specs that masked the issue ; exactly why "beat the compiler" was a required phase, not a nice-to-have
32	The first full-provenance run claimed **p23_popcnt_bytes 16 → 14** ; but three of the 14 forms were `andq $imm64` (`$0x5555555555555555`, `$0xcccccccccccccccd`, `$0x3333333333333333`), which real x86 cannot encode: AND takes sign-extended imm32 only, which is exactly why gcc materialized those constants with movabsq.	inspection of the claimed program against x86 encoding rules	the fifth instance of model-vs-world: a verifier-perfect equivalence proof over a universe that quietly contains forms the hardware doesn't have.
33	The second full-provenance run claimed **p13_sign 5 → 4** ; a genuine four-instruction sign function (`negq; setl; movq $-1, %rsp; cmovgq`) except that it materialized its -1 **in %rsp**: in-model perfectly correct (v0 treats all 16 GPRs as equal and the live-out was RAX only, and z3 proved the equivalence) but a real function body that sets RSP=-1 jumps through the return address on `ret` ; it is not a drop-in replacement for gcc's function at all.	inspection of the new claimed program against the ABI (the same discipline as row 32)	the sixth instance, and the second in one run: the model's register file is an abstraction, and "equivalent, live-out RAX" quietly diverges from "equivalent function" whenever the winner touches RSP/RBP/RBX/R12-R15.
34	`audit_claims.py` printed its **AUDIT PASSED verdict mid-file** ; right after the optimize section ; and `sys.exit(0)`-by-default meant every check added after that point (all of the compiler-batch sections) could print `FAIL` lines without changing the exit code.	extending the audit for the full batch (its own probe output)	the same failure family as row 28, one layer up: the *verifier of the numbers* had a number of its own that didn't decompose to what it checked.

[2] R. Joshi, G. Nelson, and K. Randall, "Denali: A goal-directed superoptimizer," in *Proc. Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS X)*, 2002, pp. 304–314.

[3] S. Gulwani, S. Jha, A. Tiwari, and R. Venkatesan, "Synthesis of loop-free programs," in *Proc. 32nd ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011, pp. 62–73.

[4] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic superoptimization of x86 binaries," in *Proc. 18th Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS XVIII)*, 2013, pp. 305–316.

[5] H. S. Warren, *Hacker's Delight*, 2nd ed. Addison-Wesley, 2013.

[6] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008, pp. 337–340.

[7] System V Application Binary Interface, AMD64 Architecture Processor Supplement, Draft 1.0, AMD64 Systems Community, 2023.

[8] Intel 64 and IA-32 Architectures Software Developer's Manual, Combined Volumes 1–4, Intel Corp., 2024.

[9] S. Bansal and A. Aiken, "Automatic generation of peephole superoptimizers," in *Proc. 12th Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)*, 2006.

[10] S. Bansal and A. Aiken, "Binary translation using peephole superoptimizers," in *Proc. 8th USENIX Symp. on Operating Systems Design and Implementation (OSDI)*, 2008, pp. 177–192.

[11] R. Sasnauskas, Y. Chen, P. Collingbourne, J. Ketema, G. Lup, J. Taneja, and J. Regehr, "Souper: A synthesizing superoptimizer," arXiv:1711.04422, 2017.

[12] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 1384, 1998, pp. 151–166.

[13] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proc. 35th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2014.

[14] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, "Alive2: Bounded translation validation for LLVM," in *Proc. 42nd ACM SIGPLAN Int'l Conf. on Programming Language Design and Implementation (PLDI)*, 2021.

[15] S. Dasgupta, D. Park, T. Kasampalis, V. S. Adve, and G. Roşu, "A complete formal semantics of x86-64 user-level instruction set architecture," in *Proc. 40th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2019.

[16] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic optimization of floating-point programs with tunable precision," in *Proc. 35th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2014.

[17] P. M. Phothilimthana *et al.*, "Chlorophyll: Synthesis-aided compiler for low-power spatial architectures," in *Proc. 35th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2014, pp. 396–407.

[18] R. Alur, R. Bodík, G. Juniwal, M. M. K. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa, "Syntax-guided synthesis," in *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, 2013.

[19] A. Solar-Lezama, L. Tancau, R. Bodik, S. Seshia, and V. Saraswat, "Combinatorial sketching for finite programs," in *Proc. 12th Int'l Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII)*, 2006.

[20] E. Torlak and R. Bodik, "A lightweight symbolic virtual machine for solver-aided host languages," in *Proc. 35th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2014.

[21] O. Polozov and S. Gulwani, "FlashMeta: A framework for inductive program synthesis," in *Proc. ACM SIGPLAN Int'l Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2015.

[22] S. Gulwani, O. Polozov, and R. Singh, "Program synthesis," *Foundations and Trends in Programming Languages*, vol. 4, no. 1–2, pp. 1–119, 2017.

[23] C. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanović, T. King, A. Reynolds, and C. Tinelli, "CVC4," in *Proc. 23rd Int'l Conf. on Computer Aided Verification (CAV)*, 2011.

[24] H. Barbosa *et al.*, "cvc5: A versatile and industrial-strength SMT solver," in *Proc. 28th Int'l Conf. on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2022.

[25] S. K. Jha, S. A. Seshia, and R. Somenzi, "Beaver: Engineering an efficient SMT solver for bit-vector arithmetic," in *Proc. 21st Int'l Conf. on Computer Aided Verification (CAV)*, LNCS 5643, 2009, pp. 668–674.

[26] A. Stump, C. W. Barrett, D. L. Dill, and J. Levitt, "A decision procedure for an extensional theory of arrays," in *Proc. 16th Annual IEEE Symp. on Logic in Computer Science (LICS)*, 2001.

[27] B. Dutertre and L. de Moura, "A fast linear-arithmetic solver for DPLL(T)," in *Proc. 18th Int'l Conf. on Computer Aided Verification (CAV)*, LNCS 4144, 2006.

[28] R. Nieuwenhuis, A. Oliveras, and C. Tinelli, "Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T)," *J. ACM*, vol. 53, no. 6, pp. 937–977, 2006.

[29] M. Bozzano, R. Bruttomesso, A. Cimatti, T. Junttila, P. van Rossum, S. Schulz, and R. Sebastiani, "MathSAT: Tight integration of SAT and mathematical decision procedures," *J. Automated Reasoning*, vol. 35, pp. 265–293, 2005.

[30] C. Barrett, R. Sebastiani, S. A. Seshia, and C. Tinelli, "Satisfiability modulo theories," in *Handbook of Satisfiability*, 2nd ed. IOS Press, 2021, ch. 33, pp. 1267–1329.

[31] D. Kroening and O. Strichman, *Decision Procedures: An Algorithmic Point of View*. Springer, 2008.

[32] G. C. Necula, "Translation validation for an optimizing compiler," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2000.

[33] X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[34] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. 32nd ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2011.

[35] J. Zhao, S. Nagarakatte, M. M. K. Martin, and S. Zdancewic, "Formalizing the LLVM intermediate representation for verified program transformations," in *Proc. 39th ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, 2012, pp. 427–440.

[36] S. Goel, W. A. Hunt, Jr., and M. Kaufmann, "Engineering a formal, executable x86 ISA simulator for software verification," in *Provably Correct Systems*. Springer, 2017, pp. 173–209.

[37] P. Sewell, S. Sarkar, S. Owens, F. Z. Nardelli, and M. O. Myreen, "x86-TSO: A rigorous and usable programmer's model for x86 multiprocessors," *Commun. ACM*, vol. 53, no. 7, pp. 89–97, 2010.

[38] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[39] A. Groce, G. Holzmann, and R. Joshi, "Randomized differential testing as a prelude to formal verification," in *Proc. 29th Int'l Conf. on Software Engineering (ICSE)*, 2007.

[40] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, and X. Yang, "Test-case reduction for C compiler bugs," in *Proc. 33rd ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2012.

[41] Y. Chen, A. Groce, C. Zhang, W.-K. Wong, X. Fern, E. Eide, and J. Regehr, "Taming compiler fuzzers," in *Proc. 34th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2013.

[42] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. Fifth ACM SIGPLAN Int'l Conf. on Functional Programming (ICFP)*, 2000.

[43] C. Barrett, A. Stump, and C. Tinelli, "The SMT-LIB standard: Version 2.0," Dept. of Computer Science, The University of Iowa, Tech. Rep., 2010.

[44] Advanced Micro Devices, *AMD64 Architecture Programmer's Manual, Volume 1: Application Programming*, AMD Publication 24592.

[45] A. Fog, "Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs," Technical University of Denmark, 2024.